

Module 1 : Le "Pourquoi" – Comprendre le modèle non-bloquant

1. Le problème du Synchronisme : Qu'est-ce qu'on attend ?

Dans le développement traditionnel (**synchrone**), les lignes de code s'exécutent strictement l'une après l'autre. Si la ligne 2 prend du temps, la ligne 3 attend. C'est ce qu'on appelle du code **bloquant**.

On classe les tâches informatiques en deux grandes catégories :

A. Les tâches CPU-Bound (Liées au processeur)

Le processeur (CPU) travaille activement à 100 %.

- *Exemples* : Calculer des millions de décimales de Pi, crypter un mot de passe, redimensionner une image, entraîner un modèle d'IA.
- *Impact* : On ne peut pas accélérer ce temps autrement qu'en ayant un processeur plus rapide ou en partageant le calcul sur plusieurs cœurs (Multi-processing).

B. Les tâches I/O-Bound (Liées aux Entrées/Sorties)

Le processeur ne fait **rien**. Il attend qu'un composant externe (plus lent) réponde.

- *Exemples* : Attendre la réponse d'une API web, exécuter une requête SQL sur une base de données, lire un gros fichier sur le disque dur.
- *Impact* : Le script passe 99 % de son temps à "regarder la montre". C'est **ce temps mort précis** que l'asynchronisme cherche à optimiser.

2. L'évolution des solutions : Du Synchronisme à **asyncio**

Pour comprendre comment gérer ces attentes, utilisons la métaphore d'un restaurant qui doit servir plusieurs clients.

L'approche historique : Le modèle Synchronisme (Mono-thread)

Un seul serveur pour tout le restaurant. Il prend la commande du Client 1, va en cuisine, attend de longues minutes que le plat soit cuit sans bouger, puis le sert. Le Client 2 attend dehors pendant tout ce temps.

- **En informatique** : Une seule tâche à la fois. Si le réseau est lent, toute l'application est figée.

L'approche "Ancienne génération" : Le Multi-threading classique (**import threading**)

Pour résoudre ce problème, Python proposait historiquement la classe **Thread**.

On embauche plusieurs serveurs. Si le serveur 1 attend en cuisine, le serveur 2 peut s'occuper du Client 2.

En code, cela ressemblait à ça :

```
import threading
import time

def telecharger_fichier(nom):
    print(f"Début du téléchargement {nom}")
    time.sleep(2) # Simule l'attente réseau

# On crée et on lance deux threads système en parallèle
t1 = threading.Thread(target=telecharger_fichier, args=("A",))
t2 = threading.Thread(target=telecharger_fichier, args=("B",))

t1.start()
t2.start()
```

Pourquoi on n'utilise plus trop la classe **Thread** aujourd'hui pour l'I/O ?

Bien que pratique à l'époque, la classe **Thread** a vieilli et présente trois gros défauts en Python :

1. **L'Overhead (Le poids mémoire)** : Chaque fois que tu crées un **Thread**, Python demande au Système d'Exploitation (Windows/Linux) de créer un vrai thread système. C'est lourd. Créer 50 threads consomme beaucoup de mémoire vive (RAM).
2. **Le piège du GIL (Global Interpreter Lock)** : À cause du GIL, Python ne peut exécuter qu'un seul thread à la fois sur un cœur de processeur. Le multi-threading en Python n'est donc qu'une *illusion* de parallélisme gérée par l'OS qui passe d'un thread à l'autre très vite.
3. **Les Race Conditions (L'insécurité)** : Avec les threads, l'OS peut couper l'exécution d'une fonction à n'importe quel moment pour donner la main à un autre thread. Si deux threads essaient de modifier la même variable globale en même temps, le code plante ou corrompt les données de manière totalement imprévisible.

L'approche moderne : Le modèle Asynchrone (La magie d'**asyncio**)

Un seul serveur, mais ultra-efficace. Il prend la commande du Client 1, la transmet en cuisine. Au lieu de poireauter devant les fourneaux, **il tourne immédiatement les talons** pour aller prendre la commande du Client 2. Quand le plat du Client 1 est prêt, la cuisine "bipe" le serveur, qui s'interrompt un instant pour servir le Client 1.

- **En informatique** : Un seul thread, mais aucune attente passive. Les tâches sont de simples objets Python légers (on peut en lancer 100 000 en même temps sans surcharger la RAM). De plus, le code ne passe la main à une autre tâche **que** lorsque tu l'autorises explicitement (via le mot-clé **await**). Plus aucun risque de corruption de données surprise !

3. L'Event Loop (La boucle d'événements) : Le cœur du réacteur

Derrière **asyncio**, il n'y a plus d'interventions brutales du Système d'Exploitation comme avec la classe **Thread**. À la place, il y a un mécanisme unique géré par Python : l'**Event Loop**.

Comment elle orchestre le code :

1. L'Event Loop gère une liste de tâches à accomplir.
2. Elle lance la première tâche.
3. Dès que cette tâche rencontre un mot-clé `await` (par exemple : attendre une réponse d'API), elle dit à l'Event Loop : *"Je me mets en pause, préviens-moi quand le réseau aura répondu"*.
4. L'Event Loop met cette tâche de côté et passe immédiatement à la tâche suivante dans la liste.
5. Dès que le réseau répond pour la première tâche, l'Event Loop reçoit une notification ("le bip de la cuisine") et planifie la reprise de la tâche dès qu'un créneau se libère.

Modèle Bloquant (Synchrone) :

```
Client 1 : —[Prend commande]—(Attente cuisine : BLOQUÉ)—[Sert plat]—>
Client 2 : —[Prend commande]—...
```

Modèle Non-Bloquant (Asynchrone `asyncio`) :

```
Client 1 : —[Prend commande] —[Sert plat]—>
Client 2 : —[Prend commande]...
```

Règle d'or absolue : L'Event Loop tourne sur **un seul thread**. Si, au milieu d'une tâche asynchrone, du code bloque le processeur pendant 5 secondes (comme un calcul massif), l'Event Loop s'arrête net. Plus aucune autre tâche ne peut avancer. C'est ce qu'on appelle **"bloquer la boucle"**.

Module 2 : Les Fondations – Syntaxe et cycle de vie

Objectifs pédagogiques

- Déclarer correctement des fonctions asynchrones avec `async def`.
- Comprendre le rôle et l'emplacement du mot-clé `await`.
- Manipuler le cycle de vie d'une coroutine et l'initialisation de l'Event Loop.
- Utiliser `asyncio.create_task()` pour lancer des tâches de fond non-bloquantes.

1. La Déclaration : `async def` et la notion de Coroutine

En Python synchrone, quand on appelle une fonction, son code s'exécute immédiatement. En Python asynchrone, rajouter le mot-clé `async` devant `def` change tout.

```
# Une fonction synchrone classique
def fonction_classique():
    return "Bonjour"

# Une fonction asynchrone
async def fonction_async():
    return "Bonjour"
```

```
# Testons les appels :  
print(type(fonction_classique())) # Sortie : <class 'str'>  
print(type(fonction_async()))     # Sortie : <class 'coroutine'>
```

Qu'est-ce qu'une Coroutine ?

Lorsque tu appelles `fonction_async()`, **le code à l'intérieur ne s'exécute pas**. À la place, Python renvoie un objet appelé une **coroutine**. C'est une "boîte" qui contient des instructions prêtes à être exécutées, mais qui a besoin d'être réveillée par l'Event Loop pour démarrer.

2. La Suspension : Le mot-clé `await`

Pour exécuter une coroutine et obtenir son résultat, on utilise le mot-clé `await`.

`await` fait deux choses :

1. Il **lance l'exécution** de la coroutine et attend qu'elle se termine pour récupérer sa valeur.
2. Pendant cette attente, il **met en pause** la fonction actuelle et rend le contrôle à l'Event Loop pour qu'elle puisse faire tourner d'autres tâches.

Règle syntaxique : Tu ne pouvez utiliser `await` **qu'à l'intérieur** d'une fonction `async def`. Écrire `await` au niveau global d'un script provoquera une `SyntaxError`.

3. Le point d'entrée : `asyncio.run()`

Puisqu'on ne peut pas utiliser `await` en dehors d'une fonction `async`, il faut un moyen de lancer la première fonction de notre programme. On utilise `asyncio.run()`. Elle crée l'Event Loop en coulisses, y injecte notre fonction principale, et ferme la boucle à la fin.

```
import asyncio  
  
async def recuperer_pseudo():  
    await asyncio.sleep(1) # Simule une attente réseau non-bloquante  
    return "Dev_Python_99"  
  
async def main():  
    print("Démarrage...")  
    pseudo = await recuperer_pseudo() # Le code se met en pause ici pendant 1s  
    print(f"Pseudo : {pseudo}")  
  
asyncio.run(main())
```

4. L'exécution en arrière-plan : `asyncio.create_task()`

Si on se contente de mettre `await` devant chaque fonction, notre code s'exécute de manière séquentielle (l'une après l'autre).

Pour lancer une tâche "en tâche de fond" afin qu'elle s'exécute en parallèle pendant que notre code continue d'avancer, on l'enveloppe dans une **Task** avec `asyncio.create_task()`.

Voyons la différence cruciale à l'aide d'un exemple concret : un serveur de jeu qui doit sauvegarder le profil d'un joueur (ce qui prend du temps) tout en continuant à gérer ses actions de jeu.

Exemple :

```
import asyncio

async def sauvegarde_background():
    print("💾 [Backup] Début de la sauvegarde automatique...")
    await asyncio.sleep(3) # Simule une écriture lourde sur le disque
    print("💾 [Backup] Sauvegarde terminée avec succès !")

async def main():
    print("Le joueur entre dans le donjon.")

    tache_sauvegarde = asyncio.create_task(sauvegarde_background())

    # Pendant les 3 secondes de la sauvegarde, le joueur continue de jouer !
    print("Le joueur attaque un monstre.")
    await asyncio.sleep(1)

    print("Le joueur ramasse un trésor.")
    await asyncio.sleep(1)

    print("Le joueur quitte le donjon.")

    # À la fin du programme, on s'assure que la tâche de fond est bien finie
    await tache_sauvegarde

asyncio.run(main())
```

Ce qui s'affiche dans la console :

```
Le joueur entre dans le donjon.
💾 [Backup] Début de la sauvegarde automatique...
Le joueur attaque un monstre.
Le joueur ramasse un trésor.
💾 [Backup] Sauvegarde terminée avec succès !
Le joueur quitte le donjon.
```

Gestion des flux et parallélisme

1. Exécuter en masse : `asyncio.gather()`

Mais que faire si l'on doit lancer 5, 10 ou 50 requêtes réseau en même temps ?

Utiliser `create_task` en boucle deviendrait vite lourd à gérer. C'est là qu'intervient `asyncio.gather()`. On lui passe plusieurs coroutines, il les lance toutes en simultanée, attend qu'elles soient toutes finies, et renvoie la liste de tous les résultats **dans l'ordre initial**.

Exemple : Un tableau de bord qui charge plusieurs services

```
import asyncio
import time

async def charger_meteo():
    await asyncio.sleep(2) # Simule une API météo un peu lente
    return "22°C"

async def charger_actus():
    await asyncio.sleep(1) # Simule une API d'actualités rapide
    return ["Python 3.11 est sorti", "Découverte sur Mars"]

async def main():
    debut = time.time()
    print("Chargement du tableau de bord...")

    # On lance les deux requêtes en SIMULTANÉ
    # Note : On ne met PAS "await" devant chaque fonction ici
    resultats = await asyncio.gather(charger_meteo(), charger_actus())

    # Les résultats arrivent dans une liste, dans le même ordre que l'appel
    meteo = resultats[0]
    actus = resultats[1]

    print(f"Météo reçue : {meteo}")
    print(f"Actus reçues : {actus}")
    print(f"Temps total écoulé : {time.time() - debut:.2f} secondes")

asyncio.run(main())
```

Ce qui se passe à l'écran :

```
Chargement du tableau de bord...
Météo reçue : 22°C
Actus reçues : ['Python 3.11 est sorti', 'Découverte sur Mars']
```

🕒 Temps total écoulé : 2.00 secondes

💡 **Analyse pédagogique :** En synchrone, ce code aurait pris 3 secondes (2 + 1). En asynchrone, grâce à `gather`, les deux requêtes tournent en même temps. Le temps total est égal à la tâche **la plus lente**, soit 2 secondes.

2. Premier arrivé, premier servi : `asyncio.wait()` et `FIRST_COMPLETED`

Parfois, vous lancez plusieurs tâches mais vous n'avez pas besoin d'attendre que tout le monde finisse. Vous voulez juste **le premier résultat disponible** (l'équivalent de `Promise.race()` en JS).

On utilise `asyncio.wait()` avec l'option `return_when=asyncio.FIRST_COMPLETED`.

Exemple : Interroger deux serveurs miroirs pour avoir le fichier le plus vite possible

```
import asyncio
import random

async def telecharger_depuis_serveur(nom):
    delai = random.uniform(1, 3)
    await asyncio.sleep(delai)
    return f"Fichier du {nom} (reçu en {delai:.2f}s)"

async def main():
    # 1. On crée nos tâches
    taches = {
        asyncio.create_task(telecharger_depuis_serveur("Serveur Europe")),
        asyncio.create_task(telecharger_depuis_serveur("Serveur USA"))
    }

    # 2. On attend que la PREMIÈRE tâche se termine
    faites, en_cours = await asyncio.wait(taches,
    return_when=asyncio.FIRST_COMPLETED)

    # 3. On récupère la tâche gagnante
    tache_gagnante = faites.pop()
    print(f"Gagnant : {tache_gagnante.result()}")

    # 4. Bonne pratique : On annule les autres tâches qui tournent encore pour
    rien
    for tache in en_cours:
        tache.cancel()

    asyncio.run(main())
```

4. La modernité : Les `TaskGroup` (Python 3.11+)

Depuis Python 3.11, la communauté recommande une nouvelle manière plus robuste de gérer les tâches simultanées : la programmation asynchrone structurée via **asyncio.TaskGroup**.

L'inconvénient de **gather**, c'est que si une tâche plante, les autres continuent de tourner en tâche de fond comme des "tâches fantômes". Le **TaskGroup** règle ce problème grâce au gestionnaire de contexte (**async with**). Si une tâche lève une exception, le groupe **annule automatiquement toutes les autres tâches en cours**.

Syntaxe avec un **TaskGroup** :

```
async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            # On crée les tâches directement dans le groupe
            tache1 = tg.create_task(charger_meteo())
            tache2 = tg.create_task(charger_actus())

            # Une fois sorti du bloc 'with', TOUTES les tâches sont garanties
            terminées
            print(f"Météo finale : {tache1.result()}")
            print(f"Actus finales : {tache2.result()}")

    except ExceptionGroup as eg:
        print(f"Une ou plusieurs tâches ont échoué : {eg}")

asyncio.run(main())
```

Module 4 : HTTP Asynchrone avec HTTPX & Gestion des erreurs

Objectifs pédagogiques

- Comprendre pourquoi les bibliothèques HTTP synchrones détruisent les performances d'**asyncio**.
 - Effectuer des requêtes HTTP asynchrones (GET, POST) avec le client **httpx**.
 - Gérer proprement les Timeouts et les exceptions réseau.
 - Optimiser les performances grâce à la réutilisation des connexions via un **AsyncClient**.
-

1. Le Piège Absolu : L'interdiction du code bloquant

C'est l'erreur numéro un de tout développeur qui débute en asynchrone : utiliser une bibliothèque synchrone comme **requests** ou **urllib** à l'intérieur d'une fonction **async def**.

```
# À NE JAMAIS FAIRE
async def recuperer_donnees():
```



```
# requests est synchrone. Pendant les 2 secondes de la requête,  
# TOUTE l'Event Loop est gelée. Les autres tâches attendent.  
retour = requests.get("https://api.com")  
return retour.json()
```

Pour que la boucle d'événements puisse jongler entre les tâches, il faut impérativement que l'attente réseau soit gérée par une bibliothèque qui implémente le mot-clé `await`. C'est le rôle de **HTTPX**.

2. Premières requêtes avec HTTPX

HTTPX s'installe très simplement (`pip install httpx`) et partage la même philosophie que `requests`. La seule différence, c'est que l'ouverture du client et l'envoi de la requête nécessitent les mots-clés asynchrones.

Structure de base :

```
import asyncio  
import httpx  
  
async def main():  
    # 'async with' garantit que le client ferme proprement les connexions réseau à  
    la fin  
    async with httpx.AsyncClient() as client:  
        # On met 'await' devant la requête car le réseau est une opération I/O  
        reponse = await client.get("https://httpbin.org/get")  
  
        print(f"Statut : {reponse.status_code}")  
        # Contrairement à d'autres bibliothèques (comme aiohttp), .json() est  
        synchrone chez HTTPX  
        donnees = reponse.json()  
        print(donnees)  
  
asyncio.run(main())
```

3. Gestion des Timeouts : Éviter les requêtes "fantômes"

Sur le web réel, une API peut mettre de longues secondes à répondre, voire ne jamais répondre du tout. Si vous ne configurez pas de limite de temps (Timeout), votre script peut rester bloqué indéfiniment.

HTTPX intègre son propre système de timeout, mais Python fournit également un gestionnaire global ultra-puissant : `asyncio.timeout()`.

Option A : Le timeout intégré à HTTPX (Niveau requête)

```
async with httpx.AsyncClient() as client:
    try:
        # Si le serveur met plus de 2.5 secondes à répondre, une erreur est levée
        reponse = await client.get("https://api.com", timeout=2.5)
    except httpx.TimeoutException:
        print("L'API a mis trop de temps à répondre.")
```

Option B : Le timeout global `asyncio.timeout()` (Recommandé en Python 3.11+)

Cette approche est plus puissante car elle permet d'englober un bloc entier de code (plusieurs requêtes d'affilée par exemple).

```
try:
    async with asyncio.timeout(3.0): # Limite stricte de 3 secondes pour TOUT le
        bloc
        async with httpx.AsyncClient() as client:
            rep1 = await client.get("https://api.com/service1")
            rep2 = await client.get("https://api.com/service2")
except TimeoutError:
    print("Le bloc complet a dépassé la limite de 3 secondes !")
```

4. Robustesse : Gérer les erreurs réseau dans les tâches de fond

Quand vous lancez de nombreuses requêtes en parallèle (via un `asyncio.gather` ou un `TaskGroup`), un seul plantage réseau peut faire échouer l'ensemble de votre script. Il faut savoir intercepter les erreurs sans stopper le reste des téléchargements.

Exemple : Ignorer proprement les erreurs avec `return_exceptions=True`

```
async def recuperer_url(client, url):
    reponse = await client.get(url)
    reponse.raise_for_status() # Lève une erreur si le statut est 4xx ou 5xx
    return reponse.json()

async def main():
    urls = [
        "https://httpbin.org/status/200", # Va réussir
        "https://httpbin.org/status/404", # Va lever une erreur HTTP
        "https://httpbin.org/status/200" # Va réussir
    ]

    async with httpx.AsyncClient() as client:
        taches = [recuperer_url(client, u) for u in urls]
```

```
# Le paramètre return_exceptions=True empêche le script de crasher
resultats = await asyncio.gather(*taches, return_exceptions=True)

for index, res in enumerate(resultats):
    if isinstance(res, Exception):
        print(f"Erreur sur l'URL {index} : {res}")
    else:
        print(f"Succès sur l'URL {index} !")

asyncio.run(main())
```
